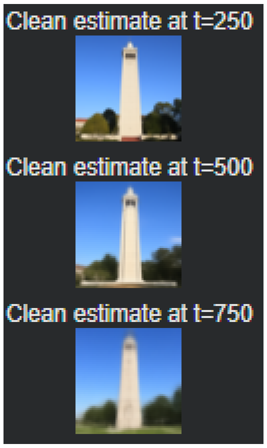
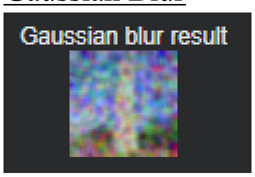
Part 1

Forward Process



The forward process adds noise to a clean image. As the timestep goes up, the image gets more and more corrupted until very little of the original picture is left. I tested noise levels at 250, 500, and 750. At 250, the lighthouse was still visible but noisy. At 500, the structure was fading. At 750, almost everything was replaced by random noise. This step shows how diffusion models purposely remove information so the model can later learn how to add it back.

Gaussian Blur



I tried using a simple Gaussian blur to “denoise” the image at t = 750. Instead of recovering anything meaningful, the blur just smoothed the noise out. This proves that traditional filters can’t reverse the diffusion process. Once real structure is gone, only a learned model can guess what should be there.

One Step Denoising

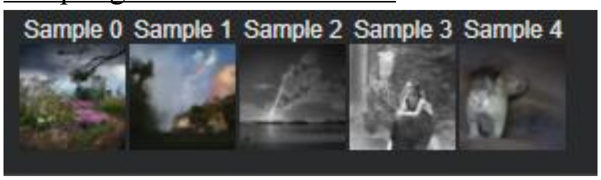
Next, I used the UNet’s noise prediction to try to denoise the image in one move. This worked okay at low noise levels, but once the image was too damaged, the single-step approach couldn’t rebuild anything detailed. This showed that denoising needs to happen in small steps, not all at once.

Iterative Denoising

To fix this, I implemented the full reverse diffusion loop. Instead of trying to restore the image in a single jump, the model gradually cleaned the image step by step. This approach worked much better and produced smoother, more structured results. Even though Stage-1 can't recreate perfect details, the improvement from one-step to iterative denoising is obvious.

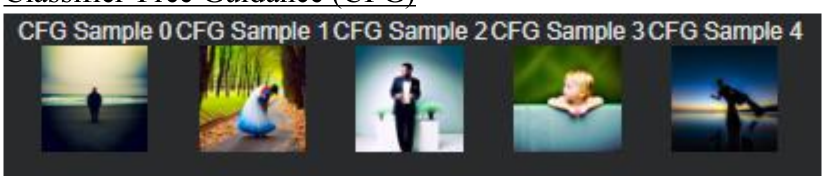
Part 2

Sampling from Random Noise



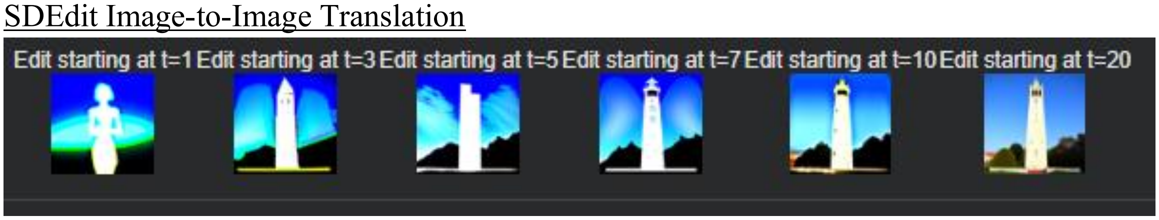
I generated images starting from pure noise by running the reverse process from the highest timestep down. The results were abstract and blurry, which is expected from Stage 1, but they still showed shapes, textures, and color patterns that prove the sampling loop works.

Classifier-Free Guidance (CFG)



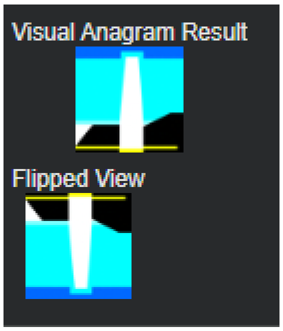
To improve the samples, I used classifier-free guidance. This method makes the model pay more attention to the text prompt by boosting the difference between the conditional and unconditional predictions. With guidance strength set to 7, the images looked sharper and more structured. This shows how guidance helps push the model toward what the prompt describes.

SDEdit Image-to-Image Translation



SDEdit involves adding noise to a real image and then partially denoising it. I tested several noise levels. Lower noise levels barely changed the image, while higher levels created more dramatic edits. This gives you control over how big or subtle the change should be. Small noise means light edits; big noise means bigger transformations.

Part 3



For this part, I created an image that looks different depending on whether you view it normally or flipped upside down. I used two prompts: one about an old man and one about people around a campfire. The trick is to take the noise prediction from the upright version and from the flipped version, flip the second prediction back, and average them. Running the reverse step on that blended prediction gives an image that changes appearance when turned upside down. Since Stage 1 can't render detailed scenes, the results are abstract, but the difference between orientations still shows that the method works.